

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.read_csv("Customer-Churn.csv")
df
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	\
0	7590-VHVEG	Female	0	Yes	No	1	
1	5575-GNVDE	Male	0	No	No	34	
2	3668-QPYBK	Male	0	No	No	2	
3	7795-CF0CW	Male	0	No	No	45	
4	9237-HQITU	Female	0	No	No	2	
...	...	...	...	...	...	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	
7039	2234-XADUH	Female	0	Yes	Yes	72	
7040	4801-JZAZL	Female	0	Yes	Yes	11	
7041	8361-LTMKD	Male	1	Yes	No	4	
7042	3186-AJIEK	Male	0	No	No	66	

	PhoneService	MultipleLines	InternetService
OnlineSecurity ... \			
0	No	No phone service	DSL
No ...			
1	Yes	No	DSL
Yes ...			
2	Yes	No	DSL
Yes ...			
3	No	No phone service	DSL
Yes ...			
4	Yes	No	Fiber optic
No ...			
...	...	...	...
...			
7038	Yes	Yes	DSL
Yes ...			
7039	Yes	Yes	Fiber optic
No ...			
7040	No	No phone service	DSL
Yes ...			
7041	Yes	Yes	Fiber optic
No ...			
7042	Yes	No	Fiber optic
Yes ...			

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
Contract \				
0	No	No	No	No
to-month				Month-

1	Yes	No	No	No	
One year					
2	No	No	No	No	Month-
to-month					
3	Yes	Yes	No	No	
One year					
4	No	No	No	No	Month-
to-month					
...	...	...	...	...	
...					
7038	Yes	Yes	Yes	Yes	
One year					
7039	Yes	No	Yes	Yes	
One year					
7040	No	No	No	No	Month-
to-month					
7041	No	No	No	No	Month-
to-month					
7042	Yes	Yes	Yes	Yes	
Two year					
PaperlessBilling		PaymentMethod		MonthlyCharges	
TotalCharges \					
0	Yes	Electronic check		29.85	
29.85					
1	No	Mailed check		56.95	
1889.5					
2	Yes	Mailed check		53.85	
108.15					
3	No	Bank transfer (automatic)		42.30	
1840.75					
4	Yes	Electronic check		70.70	
151.65					
...	...	...		...	
...					
7038	Yes	Mailed check		84.80	
1990.5					
7039	Yes	Credit card (automatic)		103.20	
7362.9					
7040	Yes	Electronic check		29.60	
346.45					
7041	Yes	Mailed check		74.40	
306.6					
7042	Yes	Bank transfer (automatic)		105.65	
6844.5					
Churn					
0	No				
1	No				

```

2      Yes
3      No
4      Yes
...    ...
7038   No
7039   No
7040   No
7041   Yes
7042   No

```

```
[7043 rows x 21 columns]
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7043 entries, 0 to 7042
```

```
Data columns (total 21 columns):
```

#	Column	Non-Null Count	Dtype
0	customerID	7043 non-null	object
1	gender	7043 non-null	object
2	SeniorCitizen	7043 non-null	int64
3	Partner	7043 non-null	object
4	Dependents	7043 non-null	object
5	tenure	7043 non-null	int64
6	PhoneService	7043 non-null	object
7	MultipleLines	7043 non-null	object
8	InternetService	7043 non-null	object
9	OnlineSecurity	7043 non-null	object
10	OnlineBackup	7043 non-null	object
11	DeviceProtection	7043 non-null	object
12	TechSupport	7043 non-null	object
13	StreamingTV	7043 non-null	object
14	StreamingMovies	7043 non-null	object
15	Contract	7043 non-null	object
16	PaperlessBilling	7043 non-null	object
17	PaymentMethod	7043 non-null	object
18	MonthlyCharges	7043 non-null	float64
19	TotalCharges	7043 non-null	object
20	Churn	7043 non-null	object

```
dtypes: float64(1), int64(2), object(18)
```

```
memory usage: 1.1+ MB
```

```
#Replacing blank space with 0
```

```
df["TotalCharges"] = df["TotalCharges"].replace(" ", "0")
```

```
df["TotalCharges"] = df["TotalCharges"].astype("float")
```

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines          7043 non-null   object
8   InternetService        7043 non-null   object
9   OnlineSecurity         7043 non-null   object
10  OnlineBackup           7043 non-null   object
11  DeviceProtection       7043 non-null   object
12  TechSupport            7043 non-null   object
13  StreamingTV            7043 non-null   object
14  StreamingMovies        7043 non-null   object
15  Contract               7043 non-null   object
16  PaperlessBilling       7043 non-null   object
17  PaymentMethod          7043 non-null   object
18  MonthlyCharges         7043 non-null   float64
19  TotalCharges           7043 non-null   float64
20  Churn                  7043 non-null   object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB

```

```
df.isnull().sum()
```

```

customerID      0
gender          0
SeniorCitizen    0
Partner          0
Dependents       0
tenure           0
PhoneService     0
MultipleLines    0
InternetService  0
OnlineSecurity   0
OnlineBackup     0
DeviceProtection 0
TechSupport      0
StreamingTV      0
StreamingMovies  0
Contract         0
PaperlessBilling 0
PaymentMethod    0
MonthlyCharges   0

```

```
TotalCharges      0
Churn              0
dtype: int64
```

```
df.isnull().sum().sum()
```

```
0
```

```
df.describe()
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692	2279.734304
std	0.368612	24.559481	30.090047	2266.794470
min	0.000000	0.000000	18.250000	0.000000
25%	0.000000	9.000000	35.500000	398.550000
50%	0.000000	29.000000	70.350000	1394.550000
75%	0.000000	55.000000	89.850000	3786.600000
max	1.000000	72.000000	118.750000	8684.800000

```
df.duplicated().sum()
```

```
0
```

```
def conv(value):
    if(value==1):
        return "yes"
    else:
        return "no"
df["SeniorCitizen"]=df["SeniorCitizen"].apply(conv)
```

#Converted 0 and 1 to YES/NO for better understanding

```
df.head(10)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure
0	7590-VHVEG	Female	no	Yes	No	1
1	5575-GNVDE	Male	no	No	No	34
2	3668-QPYBK	Male	no	No	No	2
3	7795-CF0CW	Male	no	No	No	45
4	9237-HQITU	Female	no	No	No	2
5	9305-CDSKC	Female	no	No	No	8
6	1452-KIOVK	Male	no	No	Yes	22

7	6713-OKOMC	Female	no	No	No	10
No						
8	7892-P00KP	Female	no	Yes	No	28
Yes						
9	6388-TABGU	Male	no	No	Yes	62
Yes						

	MultipleLines	InternetService	OnlineSecurity	...
DeviceProtection \				
0	No phone service	DSL	No	...
No				
1	No	DSL	Yes	...
Yes				
2	No	DSL	Yes	...
No				
3	No phone service	DSL	Yes	...
Yes				
4	No	Fiber optic	No	...
No				
5	Yes	Fiber optic	No	...
Yes				
6	Yes	Fiber optic	No	...
No				
7	No phone service	DSL	Yes	...
No				
8	Yes	Fiber optic	No	...
Yes				
9	No	DSL	Yes	...
No				

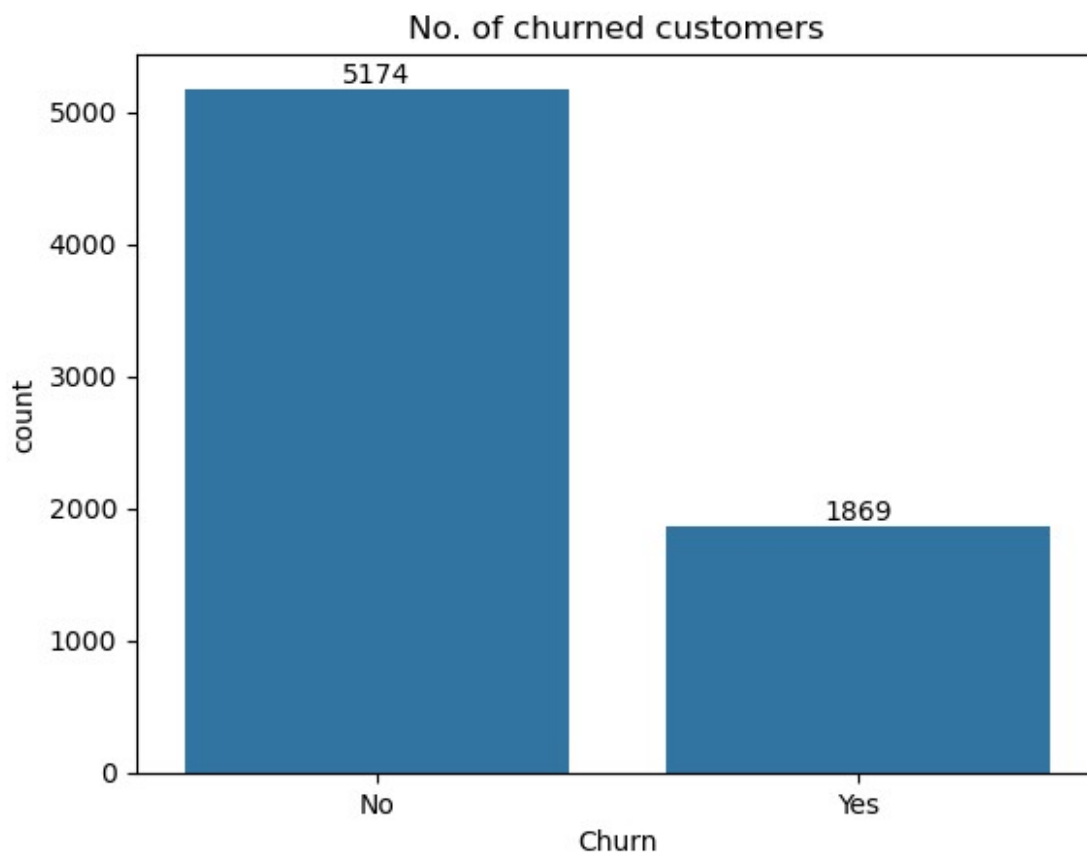
	TechSupport	StreamingTV	StreamingMovies	Contract
PaperlessBilling \				
0	No	No	No	Month-to-month
Yes				
1	No	No	No	One year
No				
2	No	No	No	Month-to-month
Yes				
3	Yes	No	No	One year
No				
4	No	No	No	Month-to-month
Yes				
5	No	Yes	Yes	Month-to-month
Yes				
6	No	Yes	No	Month-to-month
Yes				
7	No	No	No	Month-to-month
No				
8	Yes	Yes	Yes	Month-to-month

Yes					
9	No	No	No	One year	
No					

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.50	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes
5	Electronic check	99.65	820.50	Yes
6	Credit card (automatic)	89.10	1949.40	No
7	Mailed check	29.75	301.90	No
8	Electronic check	104.80	3046.05	Yes
9	Bank transfer (automatic)	56.15	3487.95	No

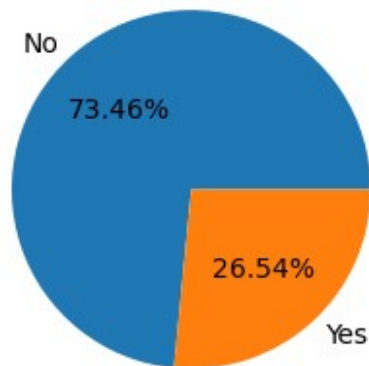
[10 rows x 21 columns]

```
ax=sns.countplot(x="Churn",data=df)
ax.bar_label(ax.containers[0])
plt.title("No. of churned customers")
plt.show()
```

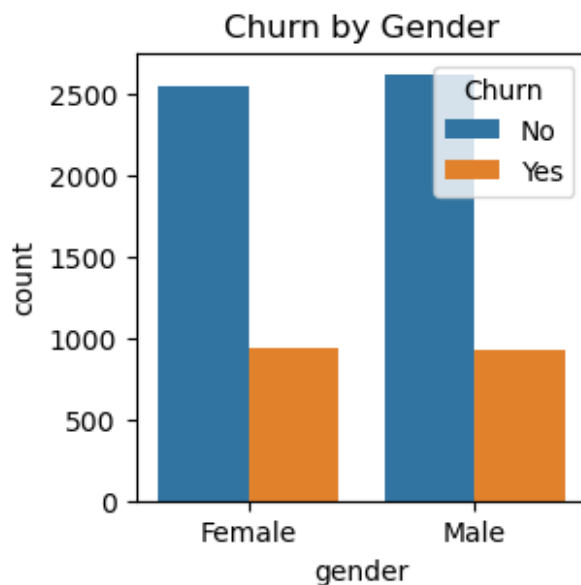


```
plt.figure(figsize=(3,3))
gb=df.groupby("Churn").agg({"Churn":"count"})
plt.pie(gb['Churn'],labels=gb.index,autopct="%1.2f%%")
plt.title("Percentage of churned customers")
plt.show()
```

Percentage of churned customers



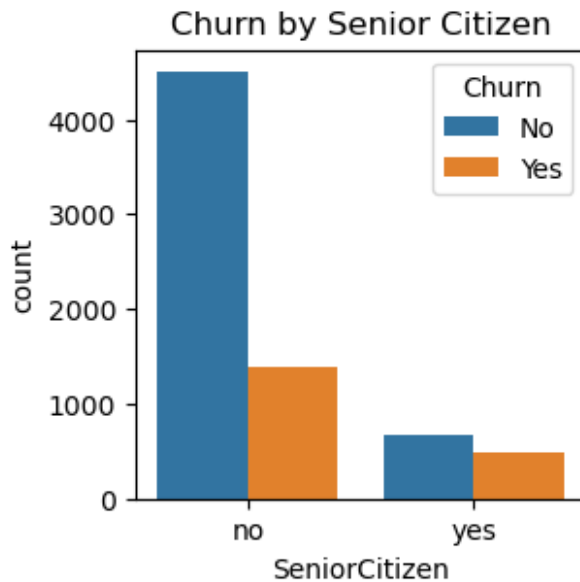
```
plt.figure(figsize=(3,3))
sns.countplot(x="gender",data=df,hue="Churn")
plt.title("Churn by Gender")
plt.show()
```



```
plt.figure(figsize=(3,3))
sns.countplot(x="SeniorCitizen",data=df,hue="Churn")
```



```
plt.title("Churn by Senior Citizen")
plt.show()
```



```
grouped = df.groupby(['SeniorCitizen',
'Churn']).size().unstack(fill_value=0)
percent = grouped.div(grouped.sum(axis=1), axis=0) * 100

# Step 2: Plotting
fig, ax = plt.subplots(figsize=(4, 4))

# Define colors (optional)
colors = ['#66c2a5', '#fc8d62'] # Customize as needed

# Plot stacked bars
bottom = [0] * len(percent)
for idx, churn_status in enumerate(percent.columns):
    bar = ax.bar(percent.index, percent[churn_status], bottom=bottom,
label=churn_status, color=colors[idx])

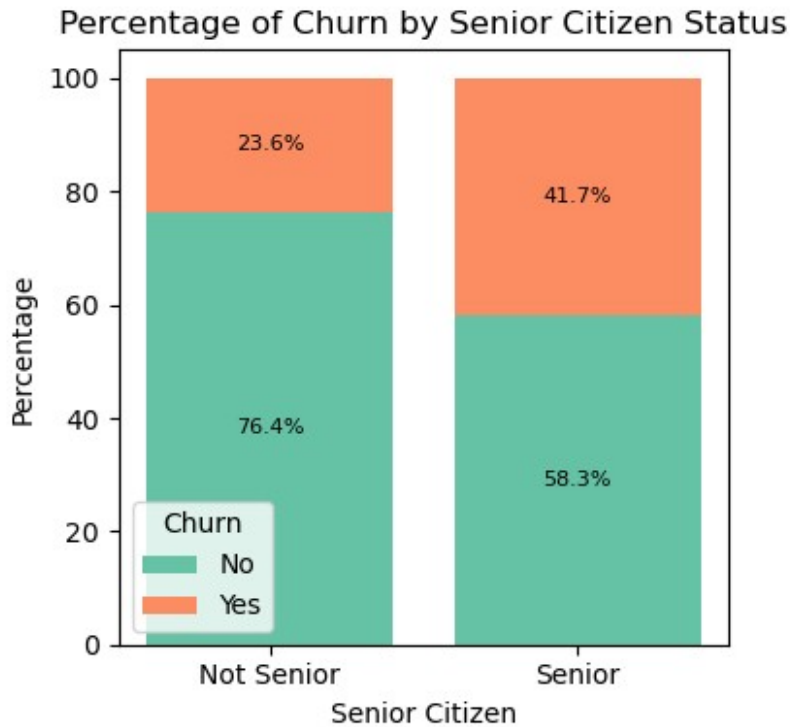
    # Annotate each segment with percentage
    for i, val in enumerate(percent[churn_status]):
        if val > 0:
            ax.text(i, bottom[i] + val / 2, f'{val:.1f}%',
ha='center', va='center', fontsize=8)
            bottom = [bottom[i] + percent[churn_status].iloc[i] for i in
range(len(bottom))]

# Step 3: Final touches
ax.set_title('Percentage of Churn by Senior Citizen Status')
ax.set_ylabel('Percentage')
ax.set_xlabel('Senior Citizen')
```

```

ax.set_xticks([0, 1])
ax.set_xticklabels(['Not Senior', 'Senior'])
ax.legend(title='Churn')
plt.tight_layout()
plt.show()

```

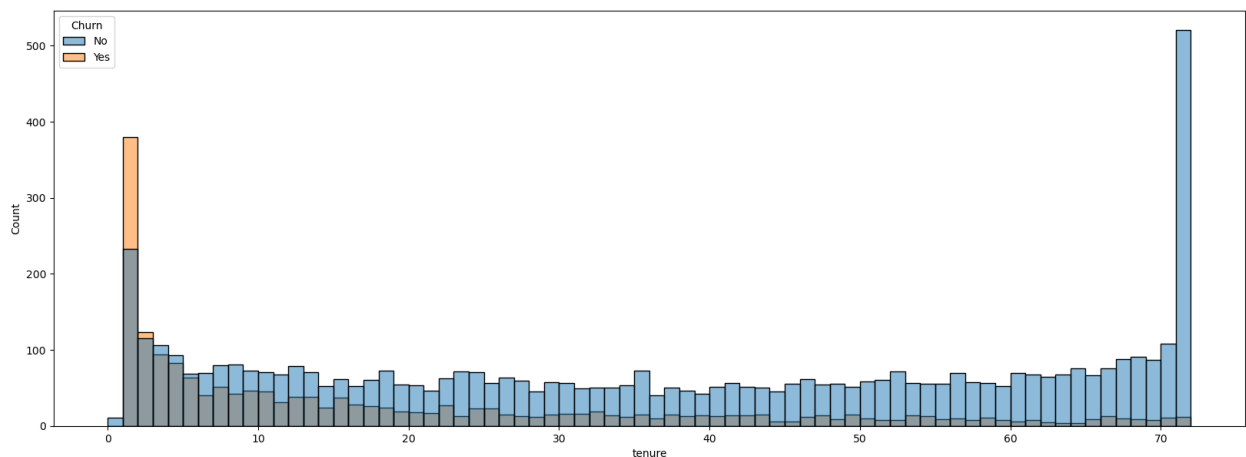


#comparatively a greater number of senior citizen people have churned

```

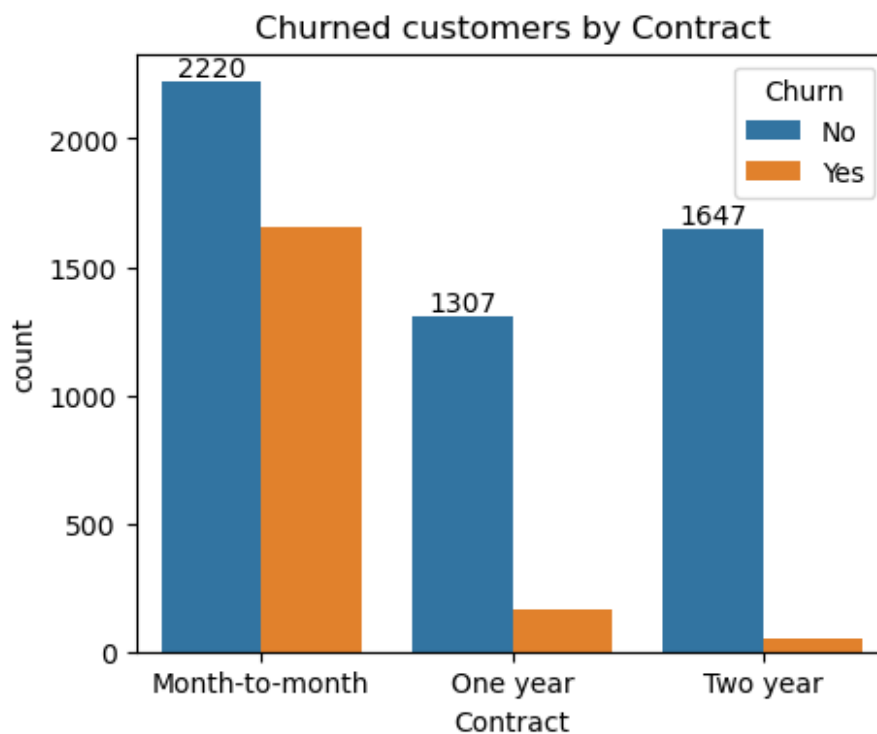
plt.figure(figsize=(20,7))
sns.histplot(x="tenure",data=df,hue="Churn",bins=72)
plt.show()

```



#People who have joined recently have churned while long tenure people have stayed

```
plt.figure(figsize=(5,4))
ax=sns.countplot(x="Contract",data=df,hue="Churn")
ax.bar_label(ax.containers[0])
plt.title("Churned customers by Contract")
plt.show()
```



#From above plot we can conclude that people having Month-to-month contract are more likely to churn than yearly contract people

```
df.columns.values
array(['customerID', 'gender', 'SeniorCitizen', 'Partner',
      'Dependents',
      'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
      'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
      'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
      'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
      'TotalCharges', 'Churn'], dtype=object)

columns = ['PhoneService', 'MultipleLines', 'InternetService',
          'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
          'TechSupport', 'StreamingTV', 'StreamingMovies']

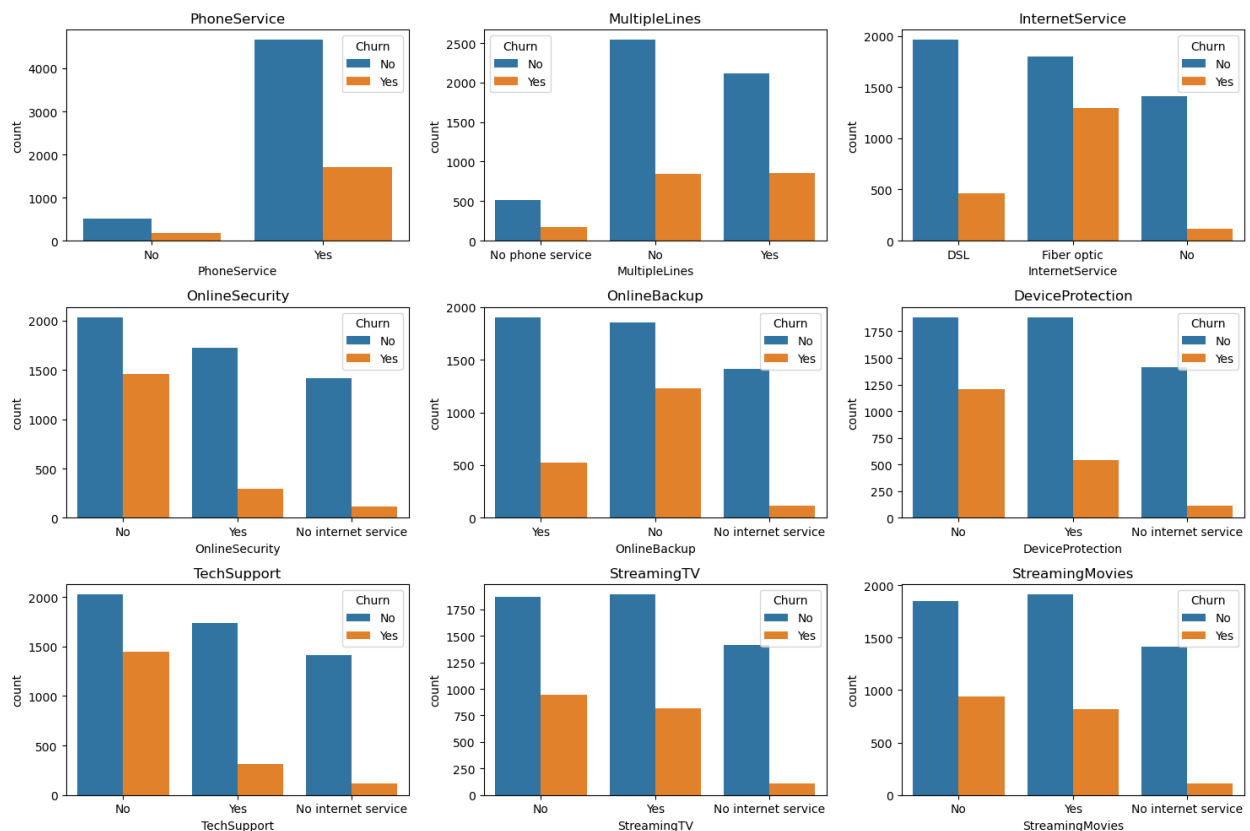
fig, axes = plt.subplots(3, 3, figsize=(15, 10))
axes = axes.flatten()
```

```

for i, col in enumerate(columns):
    sns.countplot(x=col, data=df, ax=axes[i], hue=df["Churn"])
    axes[i].set_title(col)
    axes[i].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()

```



#Customers without PhoneService or InternetService show very low churn, while those with services show a higher churn proportion.

#Features dependent on internet (like OnlineSecurity, OnlineBackup, TechSupport, etc.) tend to show higher churn when the customer has the service as "No".

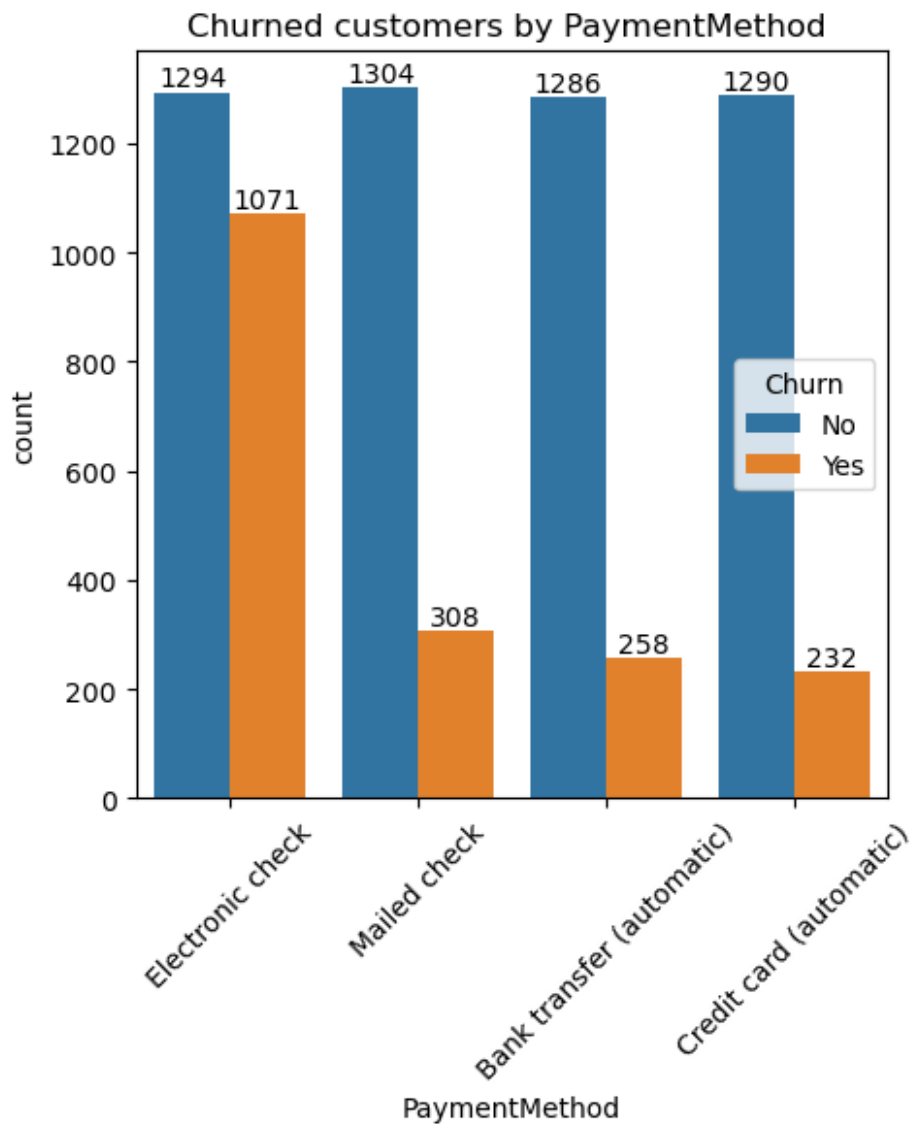
#In general, customers lacking support or protection features are more likely to churn. Streaming services (StreamingTV and StreamingMovies) also show a similar pattern, where lack of service correlates with increased churn.

```

plt.figure(figsize=(5,5))
ax=sns.countplot(x="PaymentMethod", data=df, hue="Churn")
ax.bar_label(ax.containers[0])
ax.bar_label(ax.containers[1])
plt.xticks(rotation=45)

```

```
plt.title("Churned customers by PaymentMethod ")
plt.show()
```



#Customers are more likely to churn when they use Electronic check as a payment method

Data preprocessing for ML Models

```
df.drop("customerID", axis=1, inplace=True)
df["Churn"] = df["Churn"].map({"Yes":1, "No":0})
df = pd.get_dummies(df, drop_first=True)
X = df.drop("Churn", axis=1)
y = df["Churn"]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
lr_accuracy = accuracy_score(y_test, y_pred_lr)
print("Logistic Regression Accuracy:", lr_accuracy)
print(confusion_matrix(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

Logistic Regression Accuracy: 0.8218594748048261

[[934 102]

[149 224]]

	precision	recall	f1-score	support
0	0.86	0.90	0.88	1036
1	0.69	0.60	0.64	373
accuracy			0.82	1409
macro avg	0.77	0.75	0.76	1409
weighted avg	0.82	0.82	0.82	1409

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:469: ConvergenceWarning: lbfgs failed to converge (status=1):

STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
dt_accuracy = accuracy_score(y_test, y_pred_dt)
print("Decision Tree Accuracy:", dt_accuracy)
print(confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

Decision Tree Accuracy: 0.8062455642299503

[[964 72]

[201 172]]

	precision	recall	f1-score	support
0	0.83	0.93	0.88	1036
1	0.70	0.46	0.56	373
accuracy			0.81	1409
macro avg	0.77	0.70	0.72	1409
weighted avg	0.80	0.81	0.79	1409

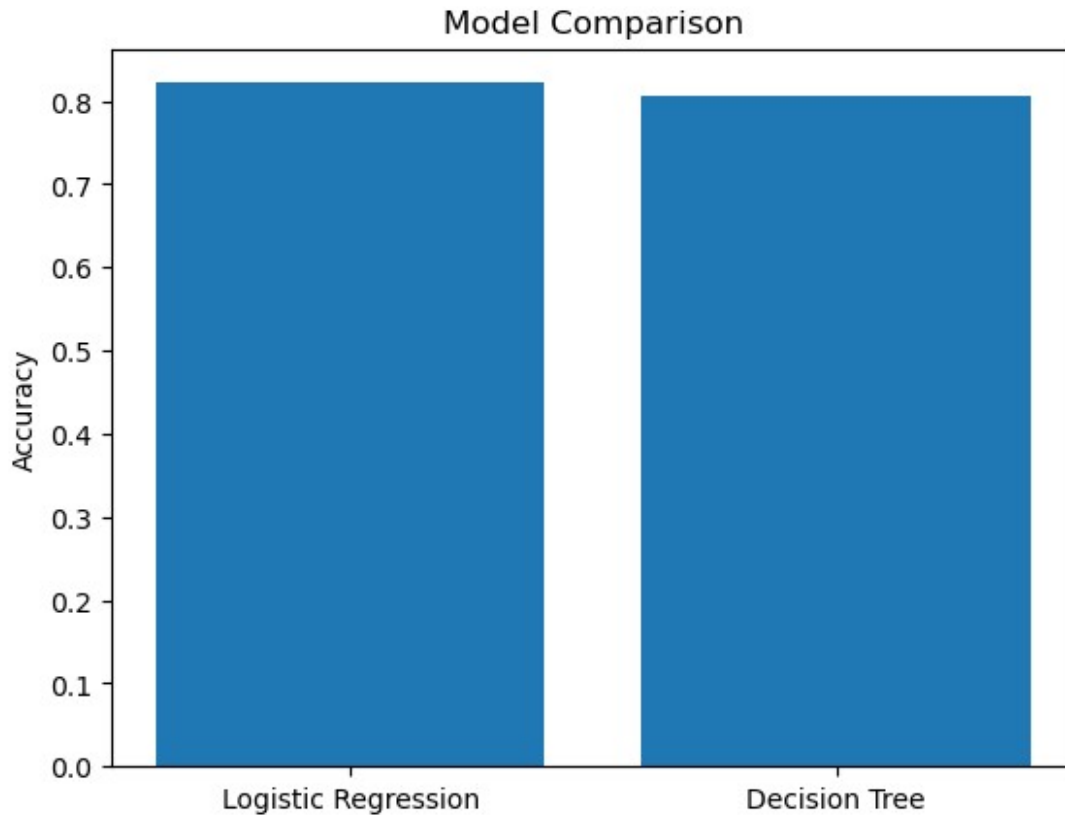
Model Comparison

```
comparison = pd.DataFrame({  
    "Model": ["Logistic Regression", "Decision Tree"],  
    "Accuracy": [lr_accuracy, dt_accuracy]  
})  
  
print(comparison)
```

	Model	Accuracy
0	Logistic Regression	0.821859
1	Decision Tree	0.806246

Accuracy Plot

```
plt.bar(comparison["Model"], comparison["Accuracy"])  
plt.title("Model Comparison")  
plt.ylabel("Accuracy")  
plt.show()
```



Feature Importance

```
feature_importance = pd.DataFrame({  
    "Feature": X.columns,  
    "Importance": dt.feature_importances_  
}).sort_values(by="Importance", ascending=False)  
  
print(feature_importance.head(10))
```

	Feature	Importance
0	tenure	0.451542
10	InternetService_Fiber optic	0.342530
2	TotalCharges	0.051739
28	PaymentMethod_Electronic check	0.027301
1	MonthlyCharges	0.022249
12	OnlineSecurity_No internet service	0.022183
24	Contract_One year	0.016530
22	StreamingMovies_No internet service	0.015881
9	MultipleLines_Yes	0.015284
25	Contract_Two year	0.012821

Logistic Coefficients

```
coeff = pd.DataFrame({  
    "Feature": X.columns,
```



```

    "Coefficient": lr.coef_[0]
}).sort_values(by="Coefficient", ascending=False)

print(coeff.head(10))

```

	Feature	Coefficient
10	InternetService_Fiber optic	0.637722
26	PaperlessBilling_Yes	0.338816
28	PaymentMethod_Electronic check	0.325492
8	MultipleLines_No phone service	0.273517
23	StreamingMovies_Yes	0.223416
9	MultipleLines_Yes	0.219068
4	SeniorCitizen_yes	0.164104
21	StreamingTV_Yes	0.124916
5	Partner_Yes	0.055738
1	MonthlyCharges	0.003066

Customer churn is primarily driven by short-term commitment, low tenure, and lack of support services. Both models consistently show that customers on month-to-month contracts, with higher charges and no value-added services, are most likely to leave.

While Logistic Regression provides reliable predictions, the Decision Tree helps uncover actionable patterns, making it easier to design targeted retention strategies.